

Travaux Pratiques - Session 10

Routes Dynamiques & Gestion des Produits

Contexte du Projet

Vous continuez le developpement de l'application **TechShop** (TP S09).

Nouvelles fonctionnalites a ajouter :

- Page de detail d'un produit (/products/:id)
- Formulaire pour ajouter un produit
- Boutons Modifier et Supprimer sur chaque produit
- Barre de recherche pour filtrer les produits
- Page 404 pour les URLs invalides

Nouveaux hooks a utiliser :

- useParams : Recuperer l'id depuis l'URL
- useNavigate : Navigation programmatique

Structure mise a jour :

```
src/  
  components/  
    layout/  
      Navbar.jsx, Footer.jsx  
  products/  
    ProductCard.jsx  
    ProductForm.jsx      <- NOUVEAU  
    SearchBar.jsx        <- NOUVEAU  
  pages/  
    Home.jsx  
    Products.jsx         <- MODIFIER  
    ProductDetail.jsx    <- NOUVEAU  
    NotFound.jsx         <- NOUVEAU  
  App.jsx                <- MODIFIER
```

1 Exercice 1 : Route Dynamique et useParams (6 pts)

Partie A : Ajouter la Route (1.5 pts)

1.1 Modifiez App.jsx pour ajouter la route dynamique vers ProductDetail :

```
// App.jsx - Ajouter cette route
<Route path="/products/:id" element={<ProductDetail />} />
```

Ecrivez le code complet de App.jsx avec toutes les routes (Home, Products, ProductDetail, NotFound) :

Partie B : Composant ProductDetail (4.5 pts)

1.2 Ecrivez le composant ProductDetail.jsx qui :

- Utilise useParams pour recuperer l'id depuis l'URL
- Fetch les donnees du produit depuis : <https://fakestoreapi.com/products/{id}>
- Gere les etats : product, loading, error
- Affiche : image (grande), titre complet, description, prix, categorie, rating
- Affiche un bouton "Retour" qui utilise useNavigate(-1)
- Affiche "Produit non trouve" si l'id n'existe pas

```
// pages/ProductDetail.jsx
// TODO: Imports (useParams, useNavigate, useState, useEffect, Link)

function ProductDetail() {
  // TODO: useParams pour recuperer id
  // TODO: useNavigate pour le bouton retour
  // TODO: useState pour product, loading, error
  // TODO: useEffect pour fetch le produit par id
  // TODO: Affichage conditionnel + rendu
}

export default ProductDetail
```

2 Exercice 2 : Lien vers Detail et Navigation (3 pts)

2.1 (1.5 pts) Modifiez ProductCard.jsx pour ajouter un lien "Voir details" vers /products/{id} :

```
// components/products/ProductCard.jsx
// Ajouter un Link dynamique vers la page detail
<Link to={`/${products}/${product.id}`}>Voir details</Link>
```

Ecrivez le composant ProductCard complet avec le lien :

2.2 (1.5 pts) Dans ProductDetail, ajoutez un bouton "Ajouter au panier" qui :

- Affiche une alerte "Produit ajoute au panier !"
- Redirige vers /products apres 1 seconde (utiliser `setTimeout` et `useNavigate`)

3 Exercice 3 : Barre de Recherche avec `filter()` (4 pts)

3.1 (1.5 pts) Ecrivez le composant `SearchBar.jsx` qui :

- Recoit `searchTerm` et `onSearchChange` en props
- Affiche un input de type text avec placeholder "Rechercher un produit..."
- Appelle `onSearchChange` a chaque changement

```
// components/products/SearchBar.jsx
function SearchBar({ searchTerm, onSearchChange }) {
  // TODO: Input controle
}
```

3.2 (2.5 pts) Modifiez `Products.jsx` pour :

- Ajouter un state `searchTerm`
- Filtrer les produits avec `filter()` selon le titre (insensible a la casse)
- Afficher `SearchBar` en haut de la page
- Afficher le nombre de resultats trouves

```
// pages/Products.jsx - Ajouter le filtrage
const [searchTerm, setSearchTerm] = useState('')

// Filtrer les produits
const filteredProducts = products.filter(product =>
  product.title.toLowerCase().includes(searchTerm.toLowerCase())
)
```

Ecrivez le code complet de `Products.jsx` avec `SearchBar` et filtrage :

4 Exercice 4 : Formulaire d'Ajout et Actions CRUD (7 pts)

Partie A : Composant `ProductForm` (3 pts)

4.1 Ecrivez le composant `ProductForm.jsx` qui :

- Recoit `onAddProduct` en props
- Gere un state local `formData` avec : title, price, category, image
- Affiche un formulaire avec les 4 champs + bouton "Ajouter"

- Appelle `onAddProduct(formData)` a la soumission
- Reinitialise le formulaire apres ajout

```
// components/products/ProductForm.jsx
function ProductForm({ onAddProduct }) {
  const [formData, setFormData] = useState({
    title: '',
    price: '',
    category: '',
    image: ''
  })

  const handleChange = (e) => {
    // TODO: Mettre a jour formData
  }

  const handleSubmit = (e) => {
    e.preventDefault()
    // TODO: Appeler onAddProduct et reinitialiser
  }

  return (
    <form onSubmit={handleSubmit}>
      {/* TODO: 4 inputs + bouton */}
    </form>
  )
}
```

Partie B : Boutons Supprimer et Modifier (2 pts)

4.2 Modifiez `ProductCard.jsx` pour ajouter :

- Un bouton "Supprimer" qui appelle `onDelete(product.id)`
- Un bouton "Modifier" qui redirige vers `/products/edit/{id}` avec `useNavigate`

```
// ProductCard.jsx - Ajouter les props onDelete
function ProductCard({ product, onDelete }) {
  const navigate = useNavigate()

  return (
    <div className="product-card">
      {/* ... contenu existant ... */}
      <div className="card-actions">
        <button onClick={() => onDelete(product.id)}>Supprimer</button>
        <button onClick={() => navigate(`/products/edit/${product.id}`)}>
          Modifier
        </button>
      </div>
    </div>
  )
}
```

```
        <Link to={`/${product.id}`}>Details</Link>
      </div>
    </div>
  )
}
```

Partie C : Integration dans Products.jsx (2 pts)

4.3 Modifiez Products.jsx pour :

- Ajouter une fonction `handleAddProduct` qui ajoute un produit au state
- Ajouter une fonction `handleDeleteProduct` qui supprime un produit par id
- Passer ces fonctions en props aux composants enfants

```
// pages/Products.jsx - Fonctions a ajouter
const handleAddProduct = (newProduct) => {
  const productWithId = {
    ...newProduct,
    id: Date.now(), // ID unique
    rating: { rate: 0, count: 0 }
  }
  setProducts([productWithId, ...products])
}

const handleDeleteProduct = (id) => {
  setProducts(products.filter(p => p.id !== id))
}
```

Ecrivez le code complet de Products.jsx avec ProductForm, SearchBar, et les actions :

5 Exercice 5 : Page 404 (Bonus - 2 pts)

5.1 Ecrivez le composant NotFound.jsx avec :

- Un titre "404 - Page non trouvee"
- Un message explicatif
- Un lien vers l'accueil
- Un bouton "Retour" avec `useNavigate(-1)`